CS 545: Cloud Native Software Engineering

Course Syllabus

Instructor

Brian Mitchell, Ph.D.
Computer Science Department
College of Computing and Informatics (CCI)
Email: bmitchell@drexel.edu

Student Learning Information

Course Description

Introduction to various aspects of software design, development and architecture used to create modern cloud computing software products. Focus will be placed on covering software engineering concepts, techniques and technologies used to build and deploy applications that run at scale on cloud infrastructure. Key topics include cloud native software engineering concepts such as working with API-driven infrastructure, software design/architecture considerations for cloud native applications, and various modern technology stacks used in creating cloud software products.

College/Department: CCI, Department of Computer Science
Repeat Status: Not repeatable for credit
Prerequisites: CS 503 [Min Grade: C]. Note that this course does not require previous background in cloud computing but does assume the student has some background in software design, software architecture and software development.

Credits: 3 hours of lecture (3 credits total)

Course Purpose within a Program of Study

This graduate-level course is an elective for MS CS and MS SE degree programs

Statement of Expected Learning

The course objectives are to:

- Study how cloud infrastructure enables software products to be deployed with resiliency and run at scale
- Concepts associated with treating the Cloud as nothing more than a "special purpose operating system"
- Explore patterns and implications associated with designing, deploying and coordinating ephemeral software components (aka containers)
- Explore cloud runtime solutions including serverless technologies and Kubernetes
- Exploring APIs including how to build them, and software engineering patterns such as event-driven architectures that are commonly used in the cloud
- Investigating common tooling, programming languages and frameworks used to create modern cloud native systems
- Investigating the 6-Pillars of well-architected cloud systems – Operational excellence, Security, Reliability, Performance efficiency, Cost optimization, and Sustainability.

As learning outcomes, students completing this course should be able to:

- Basic knowledge of programming languages used in modern cloud platforms – this class will focus on Go – aka GoLang
- Software architectures, frameworks and patterns used to construct resilient and scalable microservices and modern API based software products
- Working with Popular cloud native services such as Docker, Kubernetes, and FaaS offerings
- Have a solid understanding of cloud platform architecture concepts such as regions, availability zones, edge locations

What this course is not about:

This course is not about how to use specific cloud platforms such as AWS, GCP, or Azure. There are many excellent resources elsewhere that focus on the specifics of these cloud provider platforms, and how to interact with the services that they offer via their online portals, or via automation using tools like Terraform and/or Pulumi. While this course will have in-class demos where the instructor uses AWS and GCP to demonstrate course concepts, this course is not designed to prepare students to successfully pass cloud provider certification programs. There are many excellent resources elsewhere for these purposes – some free, and some with a modest cost. The instructor can direct you to these resources if you are interested. That said, while this course does not focus on "how to do things" using a specific cloud providers suite of services, this course instead is designed to address an orthogonal concern associated with how to design and deployed well-engineered cloud systems.

Course Time Commitment Disclosure

The time commitment required to successfully complete this course will vary significantly depending on students experience with software development toolchains, programming languages and build tools. If you have any questions or doubts about your readiness to take this course, please setup time with the instructor to discuss.

Course Materials

Required and Recommended Texts, Readings, and Resources

There is no required textbook for this course. Materials will be provided by the instructor in Blackboard, and students will be expected to review online materials (websites, Blogs, YouTube videos) provided by the instructor. The instructor will also provide a GitHub repository containing demonstration- and starter- code for various course deliverables.

Required and Supplemental Materials and Technologies

At the minimum, students should have a laptop capable and configured to run docker (https://www.docker.com/). Ideally, the student will be able to work in a local Linux environment using solutions such as the Windows Subsystem for Linux (on Windows), or via free tools like OrbStack for MacOS.

Assignments, Assessments, and Evaluations

Graded Assignments and Learning Activities

This course will balance hands on application with more traditional homework assignments. A course project will be assigned with multiple deliverables that follow the course lectures. In addition, there will be approximately 5-6 written assignments

involving reviewing material provided by the instructor (research papers, videos, etc) and answering questions based on specific prompts.

Due Dates and your Late Day Bank

This course will have assignments that will be spread out over the course as mentioned above. Given my previous experience, students tend to have different levels of readiness, especially with background in programming. I will adjust deadlines for the whole class as necessary, based on my observation of student effort and questions being asked over Discord, and attending office hours (mine or the TAs). One off deadline extensions are not possible, but I do realize that things come up from time to time. Prior to the class starting I will populate a column in Blackboard grade center called your "late day bank" with 5 late days to use over the term. Late submissions will be accepted without penalty by deducting from your late day bank. Once you exhaust your bank, late assignments will receive a 50% penalty on day 1 and will not be accepted on day 2+. Assignments are typically due at the end of the week (midnight on Friday).

Students are responsible for checking Blackboard Learn and Drexel email daily for course announcements and changes.

For more details, please refer to The Drexel University Student Handbook.

Expectations of Timely Communication for Hardships

If, for whatever reason, you find yourself struggling with keeping up with course deliverables, it is important that you let me know right away, including your plan to get back on track. I am here to help you and am willing to be as flexible as possible. However, my flexibility becomes very limited if you come to me weeks after one or more assignment deadlines have been missed with a hardship story expecting me to accept missing work without significant penalty, if even at all.

Grading Matrix

Grades will be assigned based on the following:

- Homework Assignments: 50%
- Hands on Programming Assignments (including the course project): 50%

See the course schedule section to review the various programming- and non-programming course deliverables

Grade Scale

The following scale will be used to convert points to letter grades:

Points	Grade	Points	Grade	Points	Grade
Exceptional	A+	83-86.99	B	70-72.99	C–
93-99.99	A	80-82.99	B–	67-69.99	D+
90-92.99	A–	77-79.99	C+	60-66.99	D
87-89.99	B+	73-76.99	C	0-59.99	F

Note that the instructor may revise this conversion if/when necessary.

Course Schedule

[This schedule is tentative and may change during the course.]

This course is designed to balance both theory and practical application. Most lectures will be structured with 2/3 of the content on cloud native software engineering concepts, with the remaining 1/3 being hands on tutorials and demonstrations on specific cloud native technologies.

1. Go Programming Language Tutorial – Part 1
   Tutorial: Go Programming Demonstration
   a. I/O, Conditions, Types and Interfaces, Constants, Type Alias, Arrays/Slices/Maps, Loops, Functions
   b. Go command – compile, run, test & debug
   c. Go IDE services

2. Go Programming Language Tutorial – Part 2
   Tutorial: Go Programming Demonstration
   a. Pointers, Structs, Receivers, Packages, Errors, Interfaces
   b. Go Runtime Library Overview: HTTP, API, Network Packages

3. Introduction to Cloud Computing
   a. Cloud computing models – IaaS, PaaS, FaaS
   b. Fully managed cloud services
   c. Cloud infrastructure concepts – Regions, Availability Zones
   d. Cloud native application characteristics – Scalability, Elasticity, Resiliency, Automation
   e. The big 3 cloud providers – AWS, Azure, GCP

4. Software Architectures for Cloud Native Application
   a. Event-based architectures
   b. Cloud Architecture Patterns for resiliency and scale
   c. The AWS Well Architected Framework – Part 1
   Tutorial: *API frameworks for Golang*

5. Designing Cloud Native Applications
   a. Domain Driven Design Composition
   b. Reference Architectures for Cloud Computing Products
   c. The AWS Well Architected Framework – Part 2
   Tutorial: *API Construction – Request/Reply (REST) & Event Based APIs*

6. API Deep Dive
   a. Best practices for API construction
   b. API Framework components – Routes, Middleware, Asynchronous actions Deep dive into creating Go-based APIs with Golang-Gin. Design, Deployment, Testing
   c. API Ecosystem – API Catalogs, API Gateways, API Security w/ oAuth
   Tutorial: C*reating and developing collaborating APIs*

7. Containerization & Container Orchestration
   a. Docker Architecture Overview
   b. Container Architecture Overview
   c. Practices for Building, Deploying, and managing containers
   d. Container Integration, Orchestration, and Networking
   Tutorial: Docker and Docker Compose

8. Kubernetes – Industrial Strength Container Orchestration
    a. Kubernetes Overview
    b. Kubernetes Architecture
    c. Kubernetes Cloud Provider Offerings
    Tutorial: Kubernetes (using Kubernetes KinD)

9. Function as a Service (FaaS)
    a. FaaS concepts
    b. FaaS tradeoffs vs Containers
    Tutorial: FaaS demo using AWS Lambda and AWS Gateway

10. Additional Considerations for Cloud Native Software Engineering
    a. Security Policy Management – IAM
    b. Data Considerations for Cloud Native Applications
    c. Financial Operations - FinOps
    d. Walkin topics for discussion

11. Final course project due

Course Deliverables by Week

Non-Programming Assignments

Week 2: History of Go Summary – YouTube video
Week 4: Above the Clouds Research Paper Review
Week 5: Cloud Native Software Engineering Research Paper Review
Week 8: API Design Assignment 1
Week 9: API Design Assignment 2
Weekl10: AWS Architecture Pillers Analysis Assignment

Note that the API Design Assignments will steer the APIs that you will create for the course programming project.

Programming Assignments

Week 1: Development Environment Checkout – run the Go tutorial
Week 3: Programming Assignment 1 – Building a ToDo Manager in Go

Course Programming Project

Week 6: Deliverable 1: Build a simple in memory "Voters" API
Week 7: Deliverable 2: Containerize Voter API, integrate with MySQL (or Redis)
Week 9: Deliverable 3: Build additional APIs – Votes and Polls
Week 11: Deliverable 4: Final deliverable integrating the Voters, Votes and Polls API into a Voting System. Extra credit for creating an asynchronous event manager so that voter Votes are processed using events.

Diversity Statement

CCI's vision is to nurture an inclusive community for learning and working where differences are valued, and each person is supported and empowered to reach their full potential, making CCI a powerful enabler of an equitable and just society. For more information, please see https://drexel.edu/cci/about/diversity-equity-and-inclusion-council/

Academic Policies

This course follows university, college, and department policies, including but not limited to:

- Academic Integrity, Plagiarism, Dishonesty and Cheating Policy: https://drexel.edu/provost/policies-calendars/policies/academic-integrity/
- Students with Disability Statement: https://drexel.edu/disability-resources/support-accommodations/student-family-resources/
- Course Add-Drop Policy: https://drexel.edu/provost/policies-calendars/policies/course-add-drop/
- Course Withdrawal Policy: https://drexel.edu/provost/policies-calendars/policies/course-withdrawal/
- CCI Academic Affairs Policies: https://drexel.edu/cci/current-students/policies/
- Drexel Student Learning Priorities: http://drexel.edu/provost/assessment/outcomes/dslp/

The instructor(s) may, at his/her/their discretion, change any part of the course before or during the term, including assignments, grade breakdowns, due dates, and schedule. Such changes will be communicated to students via the course web site. This web site should be checked regularly and frequently for such changes and announcements.

Students requesting accommodations due to a disability at Drexel University need to request a current Accommodations Verification Letter (AVL) in the ClockWork database before accommodations can be made. These requests are received by Disability Resources (DR), who then issues the AVL to the appropriate contacts. For additional information, visit the DR website at http://drexel.edu/oed/disabilityResources/overview/ , or contact DR for more information by phone at 215.895.1401, or by email at disability@drexel.edu.